

BOMB SWEEP

Python-Powered Game

Technical Documentation

How the Python Game Engine Works

8x8 Grid | 3 Bombs | 2 Lives | 5-Star Chocolate Prize

Built with Python, Flask, and PyScript

Author: Yashas K Gangatkar

Repository: github.com/Yashas-K-Gangatkar/TWO.LIFE

1. Project Overview

Bomb Sweep is a browser-based grid game where players must reveal all safe cells on an 8x8 board while avoiding 3 hidden bombs. The player starts with 2 lives - if they click a bomb, they lose one life. If they lose all lives, the game is over. If they successfully reveal all 61 safe cells without losing both lives, they win a 5-Star Chocolate! The entire game logic is written in Python, making up over 80% of the codebase. Python runs both on the server side (Flask) and directly in the browser (PyScript), ensuring that bomb placement, adjacent calculation, reveal logic, life deduction, and win/loss detection are all handled by Python code.

1.1 Game Rules

Rule	Details
Grid Size	8 x 8 = 64 cells
Bombs	3 bombs hidden randomly using Python random.sample()
Lives	2 lives - first bomb hit costs 1 life, second hit = Game Over
Safe Cells	61 safe cells (64 total - 3 bombs)
Adjacent Numbers	Each safe cell shows how many of its 8 neighbors are bombs
Win Condition	Reveal all 61 safe cells = 5-Star Chocolate!
Lose Condition	Hit 2 bombs (lose both lives) = Better Luck Next Time

Table 1: Game Rules Summary

1.2 Technology Stack

Component	Technology	Purpose
Game Engine	Python 3	Bomb placement, adjacent calc, reveal logic
Browser Runtime	PyScript	Runs Python in browser via WebAssembly
Server	Flask	REST API for server-mode gameplay
Android	Kotlin + WebView	Native Android wrapper for the HTML game
Frontend Shell	HTML + CSS	Minimal structure, no game logic

Table 2: Technology Stack

2. Game Engine (game/engine.py)

The core of Bomb Sweep is the `BombSweepGame` class in `game/engine.py`. This is a pure Python module with zero web dependencies. It handles all game logic: bomb placement using Python's `random.sample()`, 8-directional adjacent bomb calculation, cell reveal with life deduction, and win/loss detection. The module also includes supporting classes: `BombPlacer` for random bomb placement, `AdjacentCalculator` for counting neighboring bombs, `RevealResult` as a structured return type, and `GameStatistics` for tracking player performance across multiple games.

2.1 Bomb Placement (`BombPlacer`)

The `BombPlacer` class uses Python's built-in `random.sample()` function to uniformly distribute bombs across the grid. This ensures true randomness using Python's Mersenne Twister algorithm. The `place()` method selects 3 unique positions from the 64-cell grid, guaranteeing no duplicates. A variant method, `place_with_safe_zone()`, can exclude a specific cell to implement first-click safety, ensuring the player's first reveal is never a bomb.

Code: `BombPlacer.place()`

```
@staticmethod
def place(num_bombs=NUM_BOMBS, grid_size=GRID_SIZE):
    total = grid_size * grid_size
    if num_bombs > total:
        raise ValueError("Cannot place bombs")
    indices = random.sample(range(total), num_bombs)
    return set(indices)
```

2.2 Adjacent Bomb Calculation (`AdjacentCalculator`)

The `AdjacentCalculator` performs 8-directional neighbor checking. For each cell, it converts the flat index to (row, col) coordinates using Python's `divmod()`, then checks all 8 surrounding positions. The `calculate_all()` method pre-computes counts for all 64 cells at game start, so reveals only need a dictionary lookup. This optimization eliminates redundant calculations during gameplay, making every cell reveal instant.

Code: `AdjacentCalculator.calculate_for_cell()`

```
@staticmethod
def calculate_for_cell(cell_index, bomb_indices, grid_size=GRID_SIZE):
    row, col = divmod(cell_index, grid_size)
    count = 0
    for delta_row in (-1, 0, 1):
        for delta_col in (-1, 0, 1):
            if delta_row == 0 and delta_col == 0:
                continue
            nr, nc = row + delta_row, col + delta_col
            if 0 <= nr < grid_size and 0 <= nc < grid_size:
                neighbor_idx = nr * grid_size + nc
                if neighbor_idx in bomb_indices:
```

```
        count += 1
    return count
```

2.3 Cell Reveal Logic (BombSweepGame.reveal_cell)

The `reveal_cell()` method is the core interaction handler. When a player clicks a cell, this method determines the outcome: if the cell contains a bomb, a life is deducted; if lives reach zero, the game status changes to "lost"; if all safe cells are revealed, the status becomes "won"; otherwise, the adjacent bomb count is returned for the UI to display. The method returns a `RevealResult` object that encapsulates all information needed by both the Flask API and the PyScript UI layer, including the safe/bomb status, current lives, game status, and adjacent count.

Code: BombSweepGame.reveal_cell()

```
def reveal_cell(self, cell_index):
    if self.status != "playing":
        return RevealResult(safe=False, status=self.status,
                           lives=self.lives, error="Game not in progress")

    if cell_index in self.bomb_indices: # BOMB HIT!
        self.revealed.add(cell_index)
        self.lives -= 1
        if self.lives <= 0: # GAME OVER
            self.status = "lost"
            return RevealResult(safe=False, status="lost",
                               lives=0, bomb_index=cell_index)
        else: # Lost a life, still playing
            return RevealResult(safe=False, status="playing",
                               lives=self.lives, bomb_index=cell_index)

    # SAFE CELL
    self.revealed.add(cell_index)
    adjacent = self.adjacent_map[cell_index]
    if len(self.revealed) >= self.total_safe: # WIN!
        self.status = "won"
        return RevealResult(safe=True, status="won",
                           lives=self.lives, adjacent=adjacent)

    return RevealResult(safe=True, status="playing",
                       lives=self.lives, adjacent=adjacent)
```

2.4 Statistics Tracking (GameStatistics)

The `GameStatistics` class tracks player performance across multiple games. It records wins, losses, current and best win streaks, and calculates derived metrics like win rate percentage and average cells revealed per game. Statistics persist via `localStorage` in the browser (PyScript) or in-memory on the server (Flask). The best streak is saved so players can track their all-time best performance even after closing the browser.

3. PyScript UI Controller (pyscript_app.py)

The `pyscript_app.py` file is the bridge between the Python game engine and the browser DOM. It runs entirely in the browser via PyScript, which compiles Python to WebAssembly. This module handles all UI interactions: building the 8x8 grid by creating DOM elements, processing cell clicks, updating lives display with heart-break animations, controlling the 3-phase shuffle animation, showing modals and toasts, and triggering visual effects. Every user interaction flows through Python code before any DOM update occurs.

3.1 Grid Construction

The `build_grid()` function creates all 64 DOM elements programmatically using Python. Each cell is a `div` element with a unique ID (`cell-0` through `cell-63`), click event listener attached via PyScript's `create_proxy()`, and ARIA attributes for accessibility. The Python code creates closures for each click handler to capture the cell index, ensuring the correct cell is revealed on each click.

Code: `build_grid()`

```
def build_grid():
    grid = el("grid")
    grid.innerHTML = ""
    for i in range(TOTAL_CELLS):
        cell = document.createElement("div")
        cell.className = "cell"
        cell.id = f"cell-{i}"
        def make_handler(cell_idx):
            def handler(event=None):
                on_cell_click(cell_idx)
            return handler
        cell.addEventListener("click",
                             create_proxy(make_handler(i)))
        grid.appendChild(cell)
```

3.2 Shuffle Animation

The `shuffle_animation()` function is an async Python coroutine that orchestrates a 3-phase visual sequence before gameplay begins. Phase 1 performs a wave flip where each cell rotates in a diagonal sweep pattern repeated 3 times. Phase 2 adds random sparkle highlights using Python's `random.randint()` to select 20 random cells. Phase 3 performs a settle wave where all cells briefly shrink and bounce back. Python controls all timing through `asyncio.sleep()` and `window.setTimeout()`, with animation classes applied and removed via DOM manipulation.

3.3 Click Handler Flow

When a player clicks a cell, the `on_cell_click()` function processes the interaction entirely in Python. It first validates the click (game is in progress, cell not already revealed), then calls `game.reveal_cell()` from the Python engine. Based on the `RevealResult`, it updates the DOM: bomb hits show a bomb

emoji with a shake animation and screen flash; safe cells display the adjacent count with a flip animation and green particle burst. If the game ends, Python triggers the appropriate modal (win with celebration particles, or lose with a "Better Luck Next Time" message).

4. Flask API Server (server.py)

The Flask server provides REST API endpoints for server-mode gameplay. It creates BombSweepGame instances in memory, keyed by UUID session identifiers, and exposes endpoints for creating new games, revealing cells, revealing the full board, and retrieving aggregate statistics. The server also handles automatic cleanup of expired game sessions (older than 30 minutes) to prevent memory leaks. All game logic is delegated to the Python game engine, keeping the server layer thin and focused on HTTP routing and JSON serialization.

Endpoint	Method	Description
/api/new-game	POST	Creates a new BombSweepGame, places bombs, returns game_id
/api/reveal	POST	Reveals a cell, calls game.reveal_cell(), returns RevealResult
/api/reveal-all	POST	Returns full board data after game over
/api/stats	GET	Returns aggregate statistics across all games

Table 3: API Endpoints

Code: /api/reveal endpoint

```
@app.route("/api/reveal", methods=["POST"])
def reveal():
    data = request.get_json(silent=True)
    game_id = data.get("game_id")
    cell_index = data.get("cell_index")
    game = games[game_id]
    result = game.reveal_cell(int(cell_index))
    return jsonify(result.to_dict())
```

5. Data Flow: Click to Result

Understanding the complete data flow from a user click to a visual result is essential for maintaining and extending the game. In PyScript mode, the flow is entirely within the browser: a click event triggers the Python `on_cell_click()` function, which calls the Python game engine (`game.reveal_cell()`), receives a `RevealResult` object, and then updates the DOM directly. In Flask server mode, the click triggers a `fetch()` call to `/api/reveal`, which processes the request through the same Python game engine on the server, serializes the `RevealResult` to JSON, and returns it to the browser for DOM updates. Both paths use the identical Python game logic, ensuring consistent behavior regardless of the runtime

environment.

Step	PyScript Mode	Server Mode
1. User clicks cell	DOM click event	DOM click event
2. Handler invoked	Python <code>on_cell_click()</code>	JS <code>fetch()</code> to <code>/api/reveal</code>
3. Game logic runs	Python <code>game.reveal_cell()</code>	Python <code>game.reveal_cell()</code> on server
4. Result returned	RevealResult Python object	JSON response from Flask
5. DOM updated	Python updates DOM directly	JS updates DOM from JSON

Table 4: Data Flow Comparison - PyScript vs Server Mode

6. Project File Structure

The project is organized to maximize Python code visibility on GitHub. All game logic resides in .py files, while HTML is kept to a minimal shell with only CSS styling and structure. The `game/` directory is a proper Python package with `__init__.py` exports. The Android project wraps the HTML in a Kotlin WebView, allowing the same Python-powered game to run as a native mobile app.

File	Lines	Language	Purpose
<code>game/engine.py</code>	550	Python	Core game engine
<code>pyscript_app.py</code>	662	Python	Browser UI controller
<code>server.py</code>	235	Python	Flask API server
<code>game/__init__.py</code>	36	Python	Package exports
<code>app.py</code>	26	Python	Server entry point
<code>templates/index.html</code>	149	HTML	Minimal CSS + structure
<code>MainActivity.kt</code>	48	Kotlin	Android WebView

Table 5: Project Files - Python = 80%+ of codebase

7. How to Run

7.1 Browser Mode (No Server Needed)

The simplest way to play Bomb Sweep is to open the `templates/index.html` file directly in any modern browser. PyScript automatically loads the Python engine via WebAssembly, then loads `pyscript_app.py` and `game/engine.py`. The entire game runs in Python inside the browser - no server, no installation, no dependencies. Just double-click the HTML file and start playing. This also works on phones: send the HTML file to anyone and they can open it in their mobile browser.

7.2 Flask Server Mode

For server-based gameplay, clone the repository, install Flask, and run the server. The Flask server manages game sessions in memory with UUID keys, processes reveal requests through the same Python game engine, and provides aggregate statistics across all players. Use a virtual environment on macOS to avoid system Python conflicts: `python3 -m venv venv`, `source venv/bin/activate`, `pip install flask`, then `python3 app.py`. The server runs on port 5001 (not 5000, which conflicts with macOS AirPlay).

7.3 Android App

The `android/` directory contains a complete Android Studio project. The app uses a Kotlin Activity with a WebView that loads the game HTML from the `assets/` folder. Since PyScript loads Python from CDN, the app requires an internet connection on first launch. After that, the browser cache handles subsequent loads. To build: open the `android/` folder in Android Studio, wait for Gradle sync, then click Run. The APK can be shared directly with others for installation on their Android phones.